



PROJE RAPORU

TURUNCU PROJE

TAMAMLANDI

Theia

Yazar(lar):

Famura

İçerikler

1. Giriş.....	4
2. Bilgisayar.....	4
2.1. Proje Teknolojileri	5
2.1.1. Modülle(r)	5
2.1.2. Programlama Dilleri.....	5
2.1.3. Kütüphaneler ve Harici Bileşenler	5
2.2. Sürücü (Driver)	6
2.2.1. Sürücü Giriş Noktası	6
2.2.2. Sunucu Başlatma	7
2.2.3. Sanal Headset.....	9
2.2.4. Sanal Controller	12
2.2.5. Görüntü Yönlendirmesi	13
2.2.6. Frame Encoding	14
2.3. Kurulum Programı (Installer)	15
2.3.1. Installer Giriş Noktası ve UI Kurulumu	15
2.3.2. Installer Akışı ve Adım Sistemi.....	15
2.3.3. SteamVR Tespiti ve Doğrulama.....	17
2.3.4. Varsayılan Kurulum Dizini ve Paket Kaynağı	18
2.3.5. Gerçek Kurulum Süreci	18
2.3.6. UI Kontrolleri ve Event Yönetimi.....	19
2.4. Host.....	20
2.4.1. Giriş Noktası (Entry Point)	20
2.4.2. Host Orkestrasyonu	20
2.4.3. Renderer	22
2.4.4. Debug ve Kontrol Arayüzü	24
2.4.5. Streaming Aktarımı.....	25
2.4.6. Pose ve Tracking Durumu	26
3. Mobil	27
3.1. Project Teknolojileri	27
3.1.1. Modül(ler)	27
3.1.2. Programlama Dili	27
3.1.3. Kütüphaneler ve Harici Bileşenler	27
3.2. Uygulama Kabuk Yapısı	28
3.3. Oturum Koordinasyonu.....	28
3.4. Video Alma	29

3.5.	Video Çözme	29
3.6.	Pose Gönderimi	30
3.7.	Protokol Katmanı	31
4.	Proje Analizi	31
4.1.	Test	31
4.2.	Geliştirilebilir Şeyler	32
5.	Proje Yorumları	32
5.1.	Kullanırken Yapılacaklar	32
5.2.	Yapımcı Yorumları	32
6.	Katkıda Bulunanlar	32
7.	Referanslar	32
8.	Disk İçeriği	33

1. Giriş

Theia, Windows masaüstü ana makinesi ve Android mobil istemci etrafında inşa edilmiş iki parçalı, gerçek zamanlı bir VR (sanal gerçeklik) yayınlama projesidir. Projenin temel fikri, masaüstü tarafındaki canlı görsel çıktıyı ağ üzerinden telefona aktarmak ve aynı zamanda mobil cihazın yönelim (orientation) verisini etkileşim döngüsünün bir parçası olarak kullanmaktır.

Başka bir deyişle, Theia yalnızca bir video gönderici ya da basit bir mobil görüntüleyici değildir. Platforma özgü sorumlulukları, özel ağ iletişimi mantığı, çalışma zamanı koordinasyonu ve VR odaklı entegrasyon içeren küçük bir yayın sistemidir.

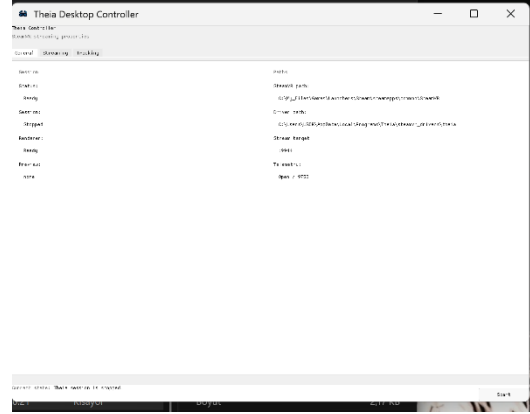
Projeyi ilginç kılan nokta, iki tarafın aynı işi tekrar etmek yerine birbirini tamamlayacak şekilde tasarlanmış olmasıdır. Masaüstü uygulaması; ana çalışma ortamını başlatma ve yönetme, yayını hazırlama ve Windows ile SteamVR tarafındaki işlemleri yürütme görevini üstlenir. Mobil uygulama ise yayını alma, verimli bir şekilde çözme (decode), ekranda gösterme ve cihaz sensörlerinden hareket verisi toplama görevlerini üstlenir. Bu ayrım projeyi modüler hale getirir ve akışı daha anlaşılır ve genişletilebilir kılar.

Proje kodu, her platformun en iyi olduğu işleri yapacak şekilde organize edilmiştir. Windows tarafı sistem entegrasyonu, render işlemleri, encoding, UDP iletim ve VR runtime yönetimiyle ilgilenir. Android tarafı ise yüzey (surface) render, donanımsal çözme (hardware decoding) ve sensör tabanlı poz (pose) yakalama işlemlerini yürütür. Bu mimari, Theia'yı tek bir özelliğe dayalı bir demo yerine gerçek bir sistem gibi hissettirir.

2. Bilgisayar

Masaüstü uygulaması, Theia'nın ana (host) tarafıdır. Çalışma ortamının başlatıldığı, yayının hazırlandığı ve SteamVR entegrasyonunun yönetildiği yerdir. Masaüstü projesi C++ ile yazılmıştır ve CMake tabanlı bir build sistemi kullanır.

Kod yapısına bakıldığında masaüstü tarafının projenin kontrol merkezi olarak tasarlandığı açıkça görülür. Uygulamanın giriş noktası Desktop/main.cpp dosyasında, çalışma zamanı yönetimi ise Desktop/src/App/Application.cpp dosyasında yer almaktadır.



Masaüstü kodu, normalde farklı araçlara veya servislere dağıtılabilecek birçok sorumluluğu bir araya getirir. Ağ bağlantısını başlatır, konfigürasyonları okur, VR runtime'ı hazırlar, render penceresini açar ve yayın oturumunu yönetir. Bu da masaüstü tarafının yalnızca "video gönderen" bir yapı olmadığını; aynı zamanda oturumun ne zaman başlayacağı, ne zaman biteceği, driver'ın nasıl kaydedileceği ve mobil istemcinin hangi adresi kullanacağı gibi kararları veren bir kontrol ortamı olduğunu gösterir.

2.1. Proje Teknolojileri

2.1.1. Mod lle(r)

- **Windows (Windows 11):**

T m proje Windows sistemlerinde  alıřacak řekilde tasarlanmıřtır. Windows 7 gibi daha eski s r mlerde test edilmemiřtir.

2.1.2. Programlama Dilleri

- **C++ (C++20 Standardı) – CLion & CMake:**

Projenin tamamı C++ ile yazılmıřtır ve C++20 standardı ile derlenecek řekilde ayarlanmıřtır. Bu, modern dil  zellikleri saęlar ve CLion ile uyumludur. CLion, IntelliJ tabanlı yapıya ařınalık ve daha kolay Git kullanımı nedeniyle tercih edilmiřtir.

2.1.3. K t phaneler ve Harici Bileřenler

- **Winsock2.h**

UDP/TCP aę iletiřimi i in Windows Sockets API saęlar.

- **ws2tcpip.h**

IP adres iřlemleri i in modern socket yardımcı fonksiyonları saęlar.

- **windows.h**

S re  oluřturma, pencere entegrasyonu ve sistem  aęrıları i in temel Windows API'yi saęlar.

- **FFmpeg**

Frame encoding, piksel format d n ř m  ve video yayını hazırlama iřlemleri i in kullanılır.

- **x264**

H.264 encoding pipeline'inin bir par ası olarak kullanılır.

- **glad**

OpenGL fonksiyon pointer'larını runtime sırasında y kler.

- **glfw3**

Pencere oluřturma, OpenGL context y netimi ve input/event iřlemleri i in kullanılır.

- **glm**

Vekt r ve matematiksel hesaplamalar i in kullanılır.

- **imgui**

Debug aray z  ve geliřtirici paneli i in kullanılır.

- **spdlog**

Loglama sistemi saęlar.

- **stb / stb_image**

Hafif g r nt  y kleme iřlemleri i in kullanılır.

- **OpenVR SDK**

SteamVR driver entegrasyonu i in kullanılır.

- **CMake**

Build sistemi ve proje yapılandırması.

- **SteamVR ara ları**

(vrpathreg.exe, vrstartup.exe) driver kaydı ve runtime bařlatma i in kullanılır.

- **CreateProcessA / taskkill.exe**

Harici s re leri bařlatmak ve sonlandırmak i in kullanılır.

- **DirectX (d3d11, dxgi)**

Grafik ve driver entegrasyonu i in kullanılır.

2.2. Sürücü (Driver)

Theia'nın **driver (sürücü)** kısmı, masaüstü uygulaması ile **SteamVR** arasında köprü görevi görür. Bu katman, projeyi basit bir video yayın uygulamasından çıkarıp gerçek bir VR entegrasyonu gibi hissettiren en önemli bileşendir.

SteamVR, Theia'nın uygulama mantığını doğrudan bilmez. Bunun yerine, özel olarak yazılmış driver modülünü yükler ve onunla **OpenVR** arayüzleri üzerinden iletişim kurar.

Driver yapısı birkaç parçaya bölünmüştür:

- **HmdDriverFactory.cpp** → Driver giriş noktası
- **TheiaServerProvider.cpp** → Sanal cihazları oluşturur ve kaydeder
- **TheiaHmdDriver.cpp** → Sanal headset (HMD)
- **TheiaControllerDriver.cpp** → Sanal controller
- **TheiaDisplayRedirectDriver.cpp** → SteamVR görüntüsünü yakalar
- **TheiaFrameEncoder.cpp** → Görüntüyü encode edip yayınlar

Bu ayrım önemli çünkü her dosya tek bir sorumluluğa sahiptir. SteamVR bunları ayrı driver objeleri olarak görür, ama aslında hepsi birlikte çalışan tek bir sistemdir.

2.2.1. Sürücü Giriş Noktası

SteamVR'ın ilk baktığı şey dışa aktarılan factory fonksiyonudur. Bu, Desktop/drivers/theia/src/HmdDriverFactory.cpp içinde uygulanmıştır. Bu, sürücünün SteamVR tarafından görünür hale geldiği noktadır.

Bu dosya oldukça küçüktür ancak tüm sürücünün başlangıç noktasıdır. SteamVR DLL'i yükler, HmdDriverFactory fonksiyonunu çağırır ve IServerTrackedDeviceProvider arayüzünü ister. Eğer istenen arayüz adı eşleşirse, fonksiyon global TheiaServerProvider örneğini döndürür. Basitçe söylemek gerekirse, bu dosya SteamVR'ın Theia sürücü koduna girmek için kullandığı kapıdır.

Burada şunlar oluyor:

- SteamVR sürücü DLL'ini yükler
- Sunucu sağlayıcı arayüzünü ister
- Factory, Theia provider nesnesini döndürür
- Bundan sonra SteamVR sürücüyü normal şekilde başlatabilir

```
THEIA_HMD_DLL_EXPORT void* HmdDriverFactory(const char* interface_name, int* return_code) {
    if (std::strcmp(interface_name, vr::IServerTrackedDeviceProvider_Version) == 0) {
        if (return_code != nullptr) {
            *return_code = vr::VRInitError_None;
        }
        return &g_server_provider;
    }

    if (return_code != nullptr) {
        *return_code = vr::VRInitError_Init_InterfaceNotFound;
    }
    return nullptr;
}
```

2.2.2. Sunucu Başlatma

Bir sonraki önemli parça Desktop/drivers/theia/src/TheiaServerProvider.cpp dosyasıdır. Bu, factory provider örneğini döndürdükten sonra SteamVR'ın çağırdığı katmandır. Init fonksiyonu, sürücünün kendini kurmaya başladığı yerdir. SteamVR sürücü context'ini alır ve ardından üç ana nesne oluşturur: TheiaHmdDriver, TheiaControllerDriver ve TheiaDisplayRedirectDriver.

```
VR_INIT_SERVER_DRIVER_CONTEXT(driver_context);

vr::VRDriverLog()->Log("Theia server provider init started");

display_redirect_ = std::make_unique<TheiaDisplayRedirectDriver>();
hmd_ = std::make_unique<TheiaHmdDriver>();
controller_ = std::make_unique<TheiaControllerDriver>(*hmd_);
hmd_->SetVirtualDisplay(static_cast<vr::IVRVirtualDisplay*>(display_redirect_.get()));
hmd_->SetControllerDriver(controller_.get());

const bool added = vr::VRServerDriverHost()->TrackedDeviceAdded(hmd_->SerialNumber().c_str(),
                                                                vr::TrackedDeviceClass_HMD,
                                                                hmd_.get());
```

Bu nesneleri oluşturduktan sonra provider bunları birbirine bağlar:

- HMD, sanal display pointer'ını alır

```
if (!added) {
    vr::VRDriverLog()->Log("Theia failed to add virtual HMD");
    hmd_.reset();
    return vr::VRInitError_Driver_Unknown;
}

char hmd_message[192] = {};
std::snprintf(hmd_message,
              sizeof(hmd_message),
              "Theia virtual HMD queued for activation serial='%s'",
              hmd_->SerialNumber().c_str());
vr::VRDriverLog()->Log(hmd_message);
```

- HMD, controller driver pointer'ını alır

```
if (!controller_added) {
    vr::VRDriverLog()->Log("Theia failed to add controller device");
    controller_.reset();
    display_redirect_.reset();
    hmd_.reset();
    return vr::VRInitError_Driver_Unknown;
}

char controller_message[224] = {};
std::snprintf(controller_message,
              sizeof(controller_message),
              "Theia controller queued for activation serial='%s'",
              controller_->SerialNumber().c_str());
vr::VRDriverLog()->Log(controller_message);
```

- Daha sonra her cihaz VRServerDriverHost()->TrackedDeviceAdded(...) ile kaydedilir

```
const bool controller_added =  
    vr::VRServerDriverHost()->TrackedDeviceAdded(controller_->SerialNumber().c_str(),  
    vr::TrackedDeviceClass_Controller,  
    controller_.get());
```

Bu sürücü mimarisinin çok önemli bir parçasıdır. SteamVR, takip edilen cihazların server host arayüzü üzerinden kaydedilmesini bekler ve Theia bu mekanizmayı kullanarak kendini sanal bir headset ve ilgili cihazlar olarak sunar.

RunFrame fonksiyonu, frame güncellemelerini bu üç cihaz nesnesine iletir. Bu, sürücünün çalışma sırasında aktif kalmasını sağlar. Her frame'de önce HMD pose'u güncellenir, sonra controller'ın pose ve input durumu güncellenir, ardından display redirect işlemleri render event'lerini ve frame yakalamayı günceller, son olarak provider SteamVR event'lerini poll eder ve dağıtır.

```
void TheiaServerProvider::RunFrame() {  
    if (hmd_ != nullptr) {  
        hmd_->RunFrame();  
    }  
    if (controller_ != nullptr) {  
        controller_->RunFrame();  
    }  
    if (display_redirect_ != nullptr) {  
        display_redirect_->RunFrame();  
    }  
  
    vr::VREvent_t event{};  
    while (vr::VRServerDriverHost()->PollNextEvent(&event, sizeof(event))) {  
        if (hmd_ != nullptr) {  
            hmd_->ProcessEvent(event);  
        }  
        if (controller_ != nullptr) {  
            controller_->ProcessEvent(event);  
        }  
        if (display_redirect_ != nullptr) {  
            display_redirect_->ProcessEvent(event);  
        }  
    }  
}
```

Bu dosya temel olarak tüm sürücünün trafik kontrolcüsüdür. İlk bakışta bu yaklaşım maliyetli gibi görünse de iyi optimizasyon ile akıcı çalışır.

2.2.3. Sanal Headset

HMD tarafı sürücünün kalbidir. Ana dosya Desktop/drivers/theia/src/TheiaHmdDriver.cpp, arayüz ise Desktop/drivers/theia/src/TheiaHmdDriver.h içinde tanımlanmıştır.

Bu sınıf, SteamVR'ın konuştuğunu düşündüğü sanal headset'i temsil eder. Headset pose'unu, display konfigürasyonunu ve etkileşim durumunu raporlamaktan sorumludur.

2.2.3.1. Aktivasyon Sırasında Olanlar

SteamVR HMD'yi aktive ettiğinde, sürücü iç pose durumunu sıfırlar, SteamVR cihaz özelliklerini doldurur, kendisini HMD olarak tanımlar, model/üretici/tracking metadata'sını ayarlar, IPD ve refresh rate gibi display değerlerini belirler ve isteğe bağlı olarak telefon pose verisi için bir UDP socket açar. Bu işlemler Activate(...) fonksiyonunda yapılır. Bu noktada headset, SteamVR açısından geçerli bir VR cihazı haline gelir.

```
vr::EVRInitError TheiaHmdDriver::Activate(const std::uint32_t object_id) {  
    object_id_ = object_id;  
    frame_number_ = 0;  
    pose_update_count_ = 0;  
    {  
        std::lock_guard<std::mutex> lock(pose_mutex_);  
        keyboard_pose_ = {};  
    }  
    phone_rotation_raw_ = IdentityQuaternion();  
    phone_rotation_offset_ = IdentityQuaternion();  
    has_phone_rotation_ = false;  
    reset_key_was_down_ = false;  
    select_key_was_down_ = false;  
    back_key_was_down_ = false;  
    interaction_active_ = false;  
    proximity_pressed_ = false;  
    has_last_phone_pose_time_ = false;  
    has_last_keyboard_update_time_ = false;  
    last_phone_pose_sequence_ = 0;  
    duplicate_phone_pose_packets_ = 0;  
    stale_phone_pose_packets_ = 0;  
    is_active = true;  
}
```

Kod ayrıca şu özellikleri ayarlar: tracking system name, model number, manufacturer, display frequency, head-to-eye depth ve IMU/proximity ile ilgili flag'ler. Bu özellikler önemlidir çünkü SteamVR cihazın nasıl davranacağını bunlara bakarak belirler.

```

const vr::PropertyContainerHandle_t container = vr::VRProperties()->TrackedDeviceToPropertyContainer(object_id);
vr::VRProperties()->SetInt32Property(container, vr::Prop_DeviceClass_Int32, vr::TrackedDeviceClass_HMD);
vr::VRProperties()->SetStringProperty(container, vr::Prop_TrackingSystemName_String, kDefaultTrackingSystem);
vr::VRProperties()->SetStringProperty(container, vr::Prop_ModelNumber_String, model_number_.c_str());
vr::VRProperties()->SetStringProperty(container, vr::Prop_ManufacturerName_String, kDefaultManufacturer);
vr::VRProperties()->SetStringProperty(container, vr::Prop_RenderModelName_String, model_number_.c_str());
vr::VRProperties()->SetFloatProperty(container, vr::Prop_UserIpDmeters_Float, display_config_.ipd_meters);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_DisplayFrequency_Float,
    display_config_.display_frequency_hz);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_UserHeadToEyeDepthMeters_Float,
    display_config_.head_to_eye_depth_meters);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_SecondsFromVsyncToPhotons_Float,
    display_config_.vsync_to_photons_seconds);
vr::VRProperties()->SetUInt64Property(container, vr::Prop_CurrentUniverseId_Uint64, 1);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_IsOnDesktop_Bool, false);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_DisplayDebugMode_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasDisplayComponent_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasVirtualDisplayComponent_Bool,
    virtual_display_ != nullptr);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_ReportsTimeSinceVsync_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_NeverTracked_Bool, false);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_DeviceHasNoIMU_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_ContainsProximitySensor_Bool, true);

```

2.2.3.2. Pose Nasıl Üretilir

HMD pose'u iki kaynaktan gelebilir: UDP üzerinden gelen telefon yönelim verisi veya klavye ile kontrol edilen fallback hareket.

Telefon yolu özellikle önemlidir. Sürücü 9955 portunda UDP socket açar ve Android uygulamasından gelen pose paketlerini dinler. Paketler geldiğinde header kontrol edilir, duplicate ve eski sequence numaraları yok sayılır. Daha sonra quaternion normalize edilir ve son telefon rotasyonu saklanır.

Ayrıca son geçerli paketin zamanı da kaydedilir.

```

vr::DriverPose_t TheiaHmdDriver::GetPose() {
    const std::uint64_t frame = frame_number_.load();
    const double bob = std::sin(static_cast<double>(frame) * 0.01) * 0.015;
    KeyboardPoseState keyboard_pose{};
    {
        std::lock_guard<std::mutex> lock(pose_mutex_);
        keyboard_pose = keyboard_pose_;
    }

    vr::DriverPose_t pose{};
    pose.qWorldFromDriverRotation = IdentityQuaternion();
    pose.qDriverFromHeadRotation = IdentityQuaternion();
    pose.qRotation = has_phone_rotation_
        ? NormalizeQuaternion(MultiplyQuaternion(phone_rotation_offset_, phone_rotation_raw_))
        : EulerToQuaternion(
            keyboard_pose.pitch_degrees,
            keyboard_pose.yaw_degrees,
            keyboard_pose.roll_degrees);
    pose.vecPosition[0] = keyboard_pose.x;
    pose.vecPosition[1] = keyboard_pose.y + bob;
    pose.vecPosition[2] = keyboard_pose.z;
    pose.result = vr::TrackingResult_Running_OK;
    pose.poseIsValid = true;
    pose.deviceIsConnected = true;
    pose.shouldApplyHeadModel = true;
    return pose;
}

```

```

if (IsKeyDown('W')) {
    keyboard_pose.x += forward_x * move_distance;
    keyboard_pose.z += forward_z * move_distance;
}
if (IsKeyDown('S')) {
    keyboard_pose.x -= forward_x * move_distance;
    keyboard_pose.z -= forward_z * move_distance;
}
if (IsKeyDown('A')) {
    keyboard_pose.x -= right_x * move_distance;
    keyboard_pose.z -= right_z * move_distance;
}
if (IsKeyDown('D')) {
    keyboard_pose.x += right_x * move_distance;
    keyboard_pose.z += right_z * move_distance;
}

```

Bu, telefonun sadece görüntüleyen bir cihaz olmadığını, aynı zamanda SteamVR tarafına yönelim verisi sağlayan bir giriş kaynağı olduğunu gösterir.

Telefon pose'u yoksa veya sorun varsa, sürücü klavye kontrolüne düşer. Kod temel klavye kontrollerini destekler: W, A, S, D hareket için; Q, E dikey hareket için; ok tuşları bakış yönü için; R reset için; Enter ve Backspace etkileşim için. Bu sayede telefon olmadan da test yapılabilir.

2.2.3.3. Her Frame'de Neler Oluyor?

RunFrame() içinde HMD: frame sayacını artırır / telefon pose paketlerini kontrol eder / klavye input durumunu kontrol eder / fallback pose'u günceller / final VR pose'u oluşturur / TrackedDevicePoseUpdated ile SteamVR'a gönderir.

```

void TheiaHmdDriver::RunFrame() {
    ++frame_number_;
    const std::uint32_t object_id = object_id_.load();
    if (is_active_ && object_id != vr::k_unTrackedDeviceIndexInvalid) {
        const auto now = std::chrono::steady_clock::now();
        const double delta_seconds = has_last_keyboard_update_time_
            ? std::chrono::duration<double>(now - last_keyboard_update_time_).count()
            : 0.0;

        last_keyboard_update_time_ = now;
        has_last_keyboard_update_time_ = true;
        PollPhonePose();
        PollKeyboardActions();
        UpdateKeyboardPose(delta_seconds);

        const vr::DriverPose_t pose = GetPose();
        vr::VRServerDriverHost()->TrackedDevicePoseUpdated(object_id, pose, sizeof(vr::DriverPose_t));

        const std::uint64_t pose_count = ++pose_update_count_;
        if (pose_count == 1 ||
            (trace_pose_updates_ && pose_trace_interval_ > 0 && pose_count % pose_trace_interval_ == 0)) {
            char message[256] = {};
            std::snprintf(message,
                sizeof(message),
                "Theia HMD pose submitted objectId=%u count=%llu valid=%d connected=%d y=%.3f result=%d",
                object_id,
                static_cast<unsigned long long>(pose_count),
                static_cast<int>(pose.poseIsValid),
                static_cast<int>(pose.deviceIsConnected),
                pose.vecPosition[1],
                static_cast<int>(pose.result));
            Log(message);
        }
    }
}

```

Bu fonksiyon, sanal headset'i SteamVR runtime'ı içinde aktif tutar. Bu şekilde SteamVR'ın 5 dakika içinde headset bağlanmazsa VR yapısını uyut yapısı bypass edilmiş olur.

2.2.3.4. . Bu Kısım Neden Önemli?

Bu sınıf, Theia'yı gerçek bir takip edilen headset gibi hissettiren parçadır. Sadece görüntü hedefi değildir; pose, input ve etkileşim durumunu da yönetir. Bu yüzden projenin en önemli parçalarından biridir.

2.2.4. Sanal Controller

Controller sürücüsü HMD'den daha küçüktür ancak yine de önemlidir. Desktop/drivers/theia/src/TheiaControllerDriver.cpp içinde bulunur. Bu sınıf SteamVR içinde sağ el controller'ı temsil eder. Fiziksel bir controller değildir; HMD pose'unu takip eden ve buton event'leri gönderebilen sanal bir controller'dır.

2.2.4.1. Peki Ne Yapıyor?

Aktivasyon sırasında sırayla: önce SteamVR controller olarak kaydedilir, sonra tracking sistemi ve model metadata'sı ayarlanır. Peşine, input profili tanımlanır ve sağ el controller olarak işaretlenir. Son olarak trigger ve application menu gibi butonlar tanımlanır.

```
vr::EVRInitError TheiaControllerDriver::Activate(const std::uint32_t object_id) {
    object_id = object_id;
    is_active_ = true;

    const vr::PropertyContainerHandle_t container = vr::VRProperties()->TrackedDeviceToPropertyContainer(object_id);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_DeviceClass_Int32, vr::TrackedDeviceClass_Controller);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_TrackingSystemName_String, kControllerTrackingSystem);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ModelNumber_String, kControllerModelNumber);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ManufacturerName_String, kControllerManufacturer);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_RenderModelName_String, kControllerRenderModel);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ControllerType_String, kControllerType);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_InputProfilePath_String, kControllerInputProfile);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_ControllerRoleHint_Int32, vr::TrackedControllerRole_RightHand);
    vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasControllerComponent_Bool, true);
    vr::VRProperties()->SetBoolProperty(container, vr::Prop_NeverTracked_Bool, false);
    vr::VRProperties()->SetUint64Property(
        container,
        vr::Prop_SupportedButtons_Uint64,
        vr::ButtonMaskFromId(vr::k_EButton_SteamVR_Trigger) |
        vr::ButtonMaskFromId(vr::k_EButton_ApplicationMenu));
    vr::VRProperties()->SetInt32Property(container, vr::Prop_Axis0Type_Int32, vr::k_eControllerAxis_None);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_Axis1Type_Int32, vr::k_eControllerAxis_Trigger);

    char message[192] = {};
    std::snprintf(message,
        sizeof(message),
        "Theia controller activated objectId=%u serial='%s'",
        object_id,
        serial_number_.c_str());
    Log(message);
    return vr::VRInitError_None;
}
```

2.2.4.2. Pose Davranışı

Controller'ın pose'u HMD'den türetilir. Headset pozisyonu ve rotasyonu alınır, bir offset uygulanır ve bu offset headset rotasyonu ile döndürülür. Böylece controller headset ile aynı uzayda durur.

2.2.4.3. Button Events

Controller şu event'leri gönderebilir:

- SetSelectPressed(...) → trigger
- SetBackPressed(...) → Uygulama menüsü

```

void TheiaControllerDriver::SetSelectPressed(const bool pressed) {
    if (pressed == select_pressed_) {
        return;
    }
    select_pressed_ = pressed;
    EmitButtonEvent(vr::k_EButton_SteamVR_Trigger, pressed);
}

void TheiaControllerDriver::SetBackPressed(const bool pressed) {
    if (pressed == back_pressed_) {
        return;
    }
    back_pressed_ = pressed;
    EmitButtonEvent(vr::k_EButton_ApplicationMenu, pressed);
}

```

Bu, denetleyicinin sanal olsa bile girdiyi SteamVR ekosistemine geri gönderebileceği anlamına gelir.

2.2.5. Görüntü Yönlendirmesi

Bu, tüm sürücünün en ilginç kısımlarından biridir. Dosya Desktop/drivers/theia/src/TheiaDisplayRedirectDriver.cpp, sınıf ise Desktop/drivers/theia/src/TheiaDisplayRedirectDriver.h içinde tanımlanmıştır.

Bu sınıf aynı anda iki işi birden yapar. Hem SteamVR sanal display arayüzünü uygular hem de render edilen backbuffer'ı yakalayıp streaming için hazırlar. Başka bir deyişle, Theia'nın SteamVR'ın ne çizdiğini "görmeye başladığı" yer burasıdır.

2.2.5.1. SteamVR Bununla Nasıl İletişim Kuruyor?

SteamVR, sanal display arayüzü üzerinden Present(...) fonksiyonunu çağırır. Bu, Theia'nın render edilen frame'i yakalamak için kullandığı hook noktasıdır.

Present(...) çağırısı geldiğinde sürücü önce gelen present bilgisini kontrol eder. Eğer bir sorun yoksa, SteamVR'dan paylaşılan backbuffer texture'ını açar. Daha sonra bu backbuffer'ı dahili bir capture texture'a kopyalar ve isteğe bağlı olarak bir preview frame yayınlar. Her şey düzgün çalışıyorsa, yakalanan texture frame encoder'a iletilir. Bu, ana render yakalama pipeline'ıdır.

```

void TheiaDisplayRedirectDriver::Present(const vr::PresentInfo_t* present_info,
                                         const std::uint32_t present_info_size) {
    const std::uint64_t count = ++present_count_;
    if (present_info != nullptr && present_info_size >= sizeof(vr::PresentInfo_t)) {
        last_present_frame_id_ = present_info->nFrameId;
        if (count == 1 ||
            (trace_virtual_display_ && trace_interval_ > 0 && (count <= 5 || count % trace_interval_ == 0))) {
            LogPresentSample(*present_info, present_info_size);
        }
        IngestBackbufferTexture(*present_info);
    } else if (count == 1 || (trace_virtual_display_ && trace_interval_ > 0 && count % trace_interval_ == 0)) {
        Log("Theia virtual display Present called with no PresentInfo");
    }
}

```

2.2.5.2. Neden D3D11 Kullanılıyor?

Sürücü burada Direct3D 11 kullanır çünkü SteamVR ve masaüstü render pipeline'ı GPU texture'ları ile çalışmaktadır. Sürücü bir D3D11 cihazı oluşturur, paylaşılan texture'ları açar ve device context üzerinden texture'lar arasında kopyalama yapar.

Bu yöntem, veriyi yüksek seviyeli bir API üzerinden manuel olarak geri okumaya kıyasla çok daha verimlidir.

```
bool TheiaDisplayRedirectDriver::EnsureD3DDevice() {
    if (d3d_device_ != nullptr) {
        return true;
    }

    constexpr D3D_FEATURE_LEVEL kFeatureLevels[] = {
        D3D_FEATURE_LEVEL_11_1,
        D3D_FEATURE_LEVEL_11_0,
        D3D_FEATURE_LEVEL_10_1,
        D3D_FEATURE_LEVEL_10_0,
    };
}
```

2.2.5.3. Preview ve Snapshot Davranışı

Display redirect driver ayrıca (kod içinde, normal kullanıcı çoğunu görmez): paylaşılan bir preview frame yayınlama / isteğe bağlı debug snapshot alma / tekrar eden capture işlemleri için trace loglama gibi işleri de destekler. Preview yolu faydalıdır çünkü encoder gerçek stream'i işlerken daha düşük frekansta bir görüntü temsilinin dışarı aktarılmasını sağlar. Debug snapshot yolu ise geliştirme sırasında faydalıdır çünkü belirli bir kopyalama sayısından sonra yakalanan belirli bir frame'in kaydedilmesini sağlar. Bu da capture sorunlarını analiz etmeye yardımcı olur.

2.2.6. Frame Encoding

Encoding katmanı Desktop/drivers/theia/src/TheiaFrameEncoder.cpp dosyasında bulunur. Bu, yakalanan D3D texture'ı ağ üzerinden gönderilebilecek bir formata dönüştüren kısımdır.

Pipeline şu şekildedir: display redirect driver'dan GPU texture alınır / taging texture'a kopyalanır / CPU belleğine map edilir / NV12 formatna dönüştürülür / Media Foundation encoder'a verilir / encode edilen çıktı paketlenir / UDP üzerinden gönderilir.

Bu sınıf önemlidir çünkü grafik yakalama ile ağ iletimi arasında bağlantı kurar. Bu olmadan display redirect driver yalnızca texture toplar, ancak bunları sistemin geri kalanına iletemez.

2.2.6.1. Encoder Ne Yapıyor?

Encoder birçok görevi vardır: frame rate'i sınırlar, böylece pipeline'ı aşırı yüklemeyi, küçük bir worker queue tutar, staging texture kopyalamayı destekler, desteklenen DXGI formatlarını CPU tarafında işlenebilir bir forma dönüştürür, frame'i stream edilebilir bir formata encode eder, video stream protokolünü kullanarak veriyi paketlere böler Bu sınıf oldukça düşük seviyeli ve performans odaklıdır. Kullanıcı arayüzüne yönelik değildir. Amacı stream'i verimli ve öngörülebilir tutmaktır.

```
bool TheiaFrameEncoder::CopyTextureToCpuFrame(ID3D11DeviceContext* context,
                                                ID3D11Texture2D* source_texture,
                                                const EncoderFrameInfo& frame_info,
                                                RawFrame& frame) {
    ID3D11Device* device = nullptr;
    source_texture->GetDevice(&device);
    const bool staging_ready = EnsureStagingTexture(device, source_texture);
    SafeRelease(device);
    if (!staging_ready || staging_texture_ == nullptr) {
        return false;
    }
}
```


2.3. Kurulum Programı (Installer)

Installer, Theia'nın paketleme ve dağıtım kısmıdır. Görevi, derlenmiş masaüstü uygulamasını ve SteamVR sürücü paketini alıp kullanıcıyı gerekli kurulum adımlarından geçirmek ve dosyaları kullanılabilir bir kurulum dizinine yerleştirmektir. Host veya driver'dan farklı olarak installer, çalışma zamanı stream'inin bir parçası değildir. Bunun yerine, kurulumdan sonra host uygulaması ve sürücünün doğru şekilde çalışabilmesi için ortamı hazırlar.

Projenin bu kısmı, Win32 API kullanılarak yerel bir Windows kurulum penceresi olarak uygulanmıştır. Installer bilinçli olarak hafif ve doğrudan olacak şekilde tasarlanmıştır: SteamVR'ın varlığını doğrular, kullanıcıdan SteamVR konumunu onaylamasını ister, paketlenmiş dosyaları kopyalar, yerel konfigürasyon dosyasını yazar ve masaüstü kısayolu oluşturur.

2.3.1. Installer Giriş Noktası ve UI Kurulumu

Ana installer mantığı Desktop/installer/src/InstallerMain.cpp dosyasındadır.

Installer, GUI uygulamaları için standart Unicode Windows giriş noktası olan wWinMain üzerinden başlar. Bu fonksiyon installer durumunu oluşturur, mümkünse SteamVR yolunu tespit eder, pencere sınıfını kaydeder, installer penceresini oluşturur ve ardından standart Windows mesaj döngüsünü çalıştırır.

Dolayısıyla installer normal bir Windows masaüstü kurulum programı gibi davranır:

- kendi penceresi vardır
- yerel (native) kontroller kullanır
- buton tıklamalarına ve metin girişlerine tepki verir
- kurulum akışı tamamlanana kadar çalışır

Pencere kaynakları ve ikon Desktop/installer/src/TheiaInstaller.rc dosyasında tanımlanmıştır. Installer'ın görsel kimliği buradan alınır.

```
int WINAPI WinMain(HINSTANCE instance, HINSTANCE previous, LPSTR, int show_command) {  
    return wWinMain(instance, previous, GetCommandLineW(), show_command);  
}
```

2.3.2. Installer Akışı ve Adım Sistemi

Installer, basit bir adım (step) tabanlı sihirbaz olarak yapılandırılmıştır. InstallerStep enum akışı tanımlar:

- Requirement
- SteamVrPath
- Confirm
- Complete

Bu yapı installer'ı hem anlaşılır hem de raporda anlatılması kolay hale getirir çünkü mantık oldukça açık şekilde ifade edilmiştir. UI, UpdateStepUi(...) fonksiyonu tarafından güncellenir ve mevcut adıma göre kontroller gösterilip gizlenir.

2.3.2.1. Adım 1: Gereksinim (Requirement)

İlk ekran kullanıcıya, devam edebilmek için SteamVR'ın önceden kurulmuş olması gerektiğini bildirir. Bu önemlidir çünkü Theia, sürücü tarafında SteamVR'a bağımlıdır.

Bu aşamada:

- sadece gereksinim mesajı gösterilir
- path input gizlidir
- browse butonu gizlidir
- yalnızca Next butonu aktiftir

Bu adım, kurulum başlamadan önce kullanıcının bağımlılığı anlamasını sağlayan bir kontrol noktasıdır.

```
std::wstring RequirementText() {  
    return L"Theia Desktop Controller kurulumuna devam etmeden önce  
    bilgisayarınızda SteamVR yüklü olmalıdır.\r\n\r\nDevam etmek için Next dugmesine basin."  
}
```

2.3.2.2. Adım 2: SteamVR Konumunu Belirleme

İkinci ekran, kullanıcının SteamVR veya Steam klasörünü seçtiği yerdir. Installer SteamVR'ı otomatik tespit etmeye çalışır, ancak gerekirse kullanıcının manuel seçim yapmasına da izin verir.

Bu adım şu fonksiyonları kullanır:

- DetectSteamVrRoot()
- NormalizeSteamVrRoot(...)
- BrowseForFolder(...)
- RefreshSteamVrValidity(...)

Installer, klasörün geçerli olduğunu doğrulamak için şu iki dosyanın varlığını kontrol eder:

- bin/win64/vrstartup.exe
- bin/win64/vrpathreg.exe

Bu dosyalar bulunursa SteamVR root geçerli kabul edilir. Kullanıcı bir klasör seçtiğinde veya yazdığında ve bu klasör geçerli bir SteamVR konumuna karşılık gelirse Next butonu aktif hale gelir.

```
if (const auto detected = DetectSteamVrRoot(); detected.has_value()) {  
    state.steamvr_root = *detected;  
    state.steamvr_valid = true;  
}
```

Bu kısım önemlidir çünkü installer girilen yolu körü körüne kabul etmez. Klasör yapısını kontrol eder ve gerçekten bir SteamVR kurulumuna işaret ettiğinden emin olur.

2.3.2.3. Adım 3: Onay (Confirm)

SteamVR doğrulandıktan sonra installer onay ekranına geçer. Bu ekranda şu bilgiler özetlenir:

- SteamVR klasörü
- kurulum dizini
- uygulama ve sürücü dosyalarının kopyalanacağı bilgisi

Bu, diske yazma işleminden önceki son kontrol noktasıdır ve kullanıcıya yapılacak işlemin net bir önizlemesini sunar.

```
std::wstring ConfirmText(const InstallerState& state) {  
    return L"Theia kurulumu hazır.\r\n\r\nSteamVR klasörü:\r\n" + state.steamvr_root.wstring() +  
        L"\r\n\r\nKurulum klasörü:\r\n" + state.install_dir.wstring() +  
        L"\r\n\r\nInstall düğmesine bastığınızda uygulama ve driver dosyaları kopyalanacak.";  
}
```

2.3.2.4. Adım 4: Tamamlanma (Complete)

Başarılı kurulumdan sonra son ekran, Theia Desktop Controller'ın kurulduğunu ve artık kurulum klasöründen veya masaüstü kısayolundan çalıştırılabileceğini bildirir.

Bu, kurulum sürecinin temiz bir şekilde sona erdiği onay aşamasıdır.

```
case InstallerStep::Complete:  
    SetControlText(window, IdTitle, L"Installation Complete");  
    SetControlText(window,  
        IdBody,  
        L"Theia Desktop Controller was installed successfully.\r\n\r\n  
        You can now launch Theia from the installed folder or desktop shortcut.");
```

2.3.3. SteamVR Tespiti ve Doğrulama

Installer'daki en önemli yardımcı fonksiyonlardan biri DetectSteamVrRoot() fonksiyonudur.

Bu fonksiyon, SteamVR'ı otomatik olarak bulmaya çalışır. Yaygın Steam kurulum konumlarını kontrol eder:

- C:\Program Files (x86)\Steam
- C:\Program Files\Steam
- proje ortamında kullanılan özel Steam konumu

Ayrıca steamapps/libraryfolders.vdf dosyasını da kontrol eder. Böylece Steam farklı bir kütüphane klasörüne kurulmuş olsa bile installer SteamVR'ı bulabilir.

Bu detay raporda önemlidir çünkü installer'ın tek bir makineye sabitlenmediğini gösterir. Paylaşılan veya farklı sistemlere taşınan projelerde daha esnek ve pratik bir yaklaşım sunar.

NormalizeSteamVrRoot(...) fonksiyonu, kullanıcı Steam root klasörünü seçtiğinde yolu daha akıllı hale getirir. Gerekirse standart steamapps/common/SteamVR yolunu ekler ve gerekli SteamVR executable'larının varlığını kontrol eder.

```
std::optional<std::filesystem::path> DetectSteamVrRoot() {
    const std::filesystem::path candidates[] = {
        LR"(C:\Program Files (x86)\Steam\steamapps\common\SteamVR)",
        LR"(C:\Program Files\Steam\steamapps\common\SteamVR)",
        LR"(C:\My_Files\Games\Launchers\Steam\steamapps\common\SteamVR)",
    };
};
```

2.3.4. Varsayılan Kurulum Dizini ve Paket Kaynağı

Installer, masaüstü uygulamasını veya sürücüyü sıfırdan derlemez. Bunun yerine build sistemi tarafından hazırlanmış bir paketi kullanır. Bu süreç Desktop/CMakeLists.txt içinde yönetilir.

CMake mantığı installer hedefini oluşturur ve build işleminden sonra şu yapıyı hazırlar: payload/app ve payload/driver/theia. Bu şekilde payload/app içine şunlar kopyalanır: Theia.exe, branding klasörü, theia.ico. Bundan sonra payload/driver/theia içine ise SteamVR sürücü paketi kopyalanır.

Bu, installer'ın build aracı değil, dağıtım aracı olarak çalışmasının temel nedenidir. Hazır bir paket üzerinden kurulum yapar.

```
if(THEIA_BUILD_DESKTOP)
    add_dependencies(TheiaInstaller Desktop)
    add_custom_command(TARGET TheiaInstaller POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E make_directory "${TARGET_FILE_DIR:TheiaInstaller}/payload/app"
        COMMAND ${CMAKE_COMMAND} -E make_directory "${TARGET_FILE_DIR:TheiaInstaller}/payload/driver/theia"
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
            "${TARGET_FILE:Desktop}"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/Theia.exe"
        COMMAND ${CMAKE_COMMAND} -E copy_directory
            "${TARGET_FILE_DIR:Desktop}/branding"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/branding"
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
            "${CMAKE_SOURCE_DIR}/assets/branding/theia.ico"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/theia.ico"
        COMMAND ${CMAKE_COMMAND} -E copy_directory
            "${THEIA_INSTALLER_DRIVER_SOURCE_DIR}"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/driver/theia"
    )
)
```

2.3.5. Gerçek Kurulum Süreci

Asıl kurulum işlemi PerformInstall(...) fonksiyonunda gerçekleşir. Bu, installer'ın en önemli fonksiyonudur.

Dosya kopyalamadan önce installer, payload'ın gerçekten var olup olmadığını kontrol eder:

- payload/app/Theia.exe
- payload/driver/theia/driver.vrdrivermanifest

Bu parçalardan biri eksikse installer durur ve hata verir. Bu, kurulumun geçerli bir paket yapısına bağlı olduğunu gösterir.

Payload doğrulandıktan sonra sırayla: hedef kurulum dizini oluşturulur, SteamVR süreçleri durdurulur, uygulama payload'ı install_dir/app içine kopyalanır, sürücü payload'ı install_dir/steamvr_drivers/theia içine kopyalanır, masaüstü konfigürasyon dosyası yazılır ve masaüstü kısayolu oluşturulur.

Bu, tam dağıtım akışıdır.

2.3.5.1. Neden Önce SteamVR Durduruluyor?

StopSteamVrProcesses() fonksiyonu dosya kopyalamadan önce çağrılır. taskkill.exe kullanarak şu süreçleri kapatır: vrmonitor.exe / vrserver.exe / vrcompositor.exe / vrdashboard.exe. Bu önemlidir çünkü SteamVR çalışırken dosyalar kilitli olabilir. Bu süreçleri durdurmak kurulum hatalarını önler.

```
void StopSteamVrProcesses() {  
    const wchar_t* process_names[] = {  
        L"vrmonitor.exe",  
        L"vrserver.exe",  
        L"vrcompositor.exe",  
        L"vrdashboard.exe",  
    };  
};
```

2.3.5.2. Neden Konfigürasyon Yazılıyor?

Kopyalama işleminden sonra installer, WriteDesktopConfig(...) ile theia_desktop.conf dosyasını yazar.

Bu konfigürasyon şunları içerir:

- phone_stream_host=127.0.0.1
- steamvr_root=...
- steamvr_driver_root=...
- stream_port=9944
- stream_fps=60
- stream_bitrate_mbps=20
- stream_udp_payload_bytes=1200

Bu dosya installer ile runtime arasında köprü görevi görür. Masaüstü uygulaması başlangıçta bu değerleri okur, dolayısıyla installer aynı zamanda runtime konfigürasyonunu da hazırlamış olur.

2.3.6. UI Kontrolleri ve Event Yönetimi

Installer penceresi, başlık etiketi, açıklama metni, path input, browse butonu, durum etiketi ve back / next / install / close butonları gibi standart kontroller içeren normal bir Win32 penceresidir. Bu kontroller WM_CREATE içinde oluşturulur ve pencere prosedürü kullanıcı etkileşimlerini WM_COMMAND üzerinden işler.

```
case WM_CREATE: {  
    auto* create_struct = reinterpret_cast<CREATESTRUCT*>(l_param);  
    state = reinterpret_cast<InstallerState*>(create_struct->lpCreateParams);  
    SetWindowLongPtrW(window, GWLP_USERDATA, reinterpret_cast<LONG_PTR*>(state));  
  
    HFONT font = static_cast<HFONT*>(GetStockObject(DEFAULT_GUI_FONT));  
  
    CreateWindowW(L"STATIC", L"", WS_CHILD | WS_VISIBLE, 24, 20, 640, 28, window,  
        reinterpret_cast<HMENU*>(IdTitle), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD | WS_VISIBLE | SS_LEFT, 24, 60, 640, 160, window,  
        reinterpret_cast<HMENU*>(IdBody), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD, 24, 210, 140, 22, window,  
        reinterpret_cast<HMENU*>(IdPathLabel), nullptr, nullptr);  
    CreateWindowExW(WS_EX_CLIENTEDGE, L"EDIT", L"", WS_CHILD | ES_AUTOHSCROLL, 24, 236, 540, 26, window,  
        reinterpret_cast<HMENU*>(IdPathEdit), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Browse...", WS_CHILD, 574, 236, 90, 26, window,  
        reinterpret_cast<HMENU*>(IdBrowseButton), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD, 24, 274, 640, 32, window,  
        reinterpret_cast<HMENU*>(IdStatusLabel), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"< Back", WS_CHILD | WS_VISIBLE, 448, 372, 96, 30, window,  
        reinterpret_cast<HMENU*>(IdBackButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Next >", WS_CHILD | WS_VISIBLE, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdNextButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Install", WS_CHILD, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdInstallButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Close", WS_CHILD, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdCloseButton), nullptr, nullptr);  
}
```

2.4. Host

Theia'nın host kısmı, SteamVR sürücüsü ile Android istemci arasında yer alan masaüstü kontrol uygulamasıdır. Bu katman; çalışma ortamını başlatmaktan, sürücüden gelen preview stream'i almaktan, frame'leri mobil cihaza iletmekten, yerel kontrol penceresini yönetmekten ve canlı durum bilgilerini göstermekten sorumludur.

Genel mimaride host, VR runtime ile ağ üzerinden yapılan yayın arasında pratik bir köprü görevi görür. Sürücü SteamVR ile iletişim kurar, ancak kullanıcının doğrudan etkileşimde bulunduğu uygulama host'tur. Masaüstü ortamını başlatır, oturumu aktif tutar ve debug arayüzü üzerinden sistemin mevcut durumunu gösterir.

2.4.1. Giriş Noktası (Entry Point)

Host uygulaması Desktop/main.cpp dosyasında başlar. Bu dosya bilinçli olarak küçük tutulmuştur. theia::app::Application nesnesini oluşturur ve çalıştırır. Windows üzerinde, uygulamanın masaüstü

```
#include "App/Application.h"

int main() {
    theia::app::Application application;
    return application.Run();
}

#ifdef _WIN32
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#include <windows.h>

int WINAPI WinMain(HINSTANCE, HINSTANCE, LPSTR, int) {
    return main();
}
#endif
```

programı olarak doğru şekilde başlatılabilmesi için WinMain giriş noktası da tanımlanmıştır.

Bu dosya kısa olmasına rağmen önemlidir çünkü asıl host mantığının giriş noktasına gömülmediğini gösterir. Bunun yerine uygulama davranışı Application sınıfında merkezleştirilmiştir. Bu yaklaşım başlangıcı temiz tutar ve runtime yönetimini kolaylaştırır.

Bu dosya: ana uygulama nesnesini oluşturur, kontrolü host runtime'a devreder, hem standart main() hem de Windows WinMain() kullanımını destekler.

2.4.2. Host Orkestrasyonu

Host tarafının asıl merkezi Desktop/src/App/Application.cpp dosyasıdır. Arayüzü Desktop/src/App/Application.h içinde tanımlanmıştır. Bu dosya her şeyi bir araya getirir. Oturumun ne zaman başlayacağını, SteamVR'in nasıl başlatılacağını, frame stream'inin nasıl yapılandırılacağını, preview receiver'ın nasıl açılacağını ve render döngüsünün frame bazında nasıl çalışacağını belirler.

Host uygulaması sadece "bir pencere gösteren" bir yapı değildir. Tüm runtime yaşam döngüsünü koordine eder. Bu süreç şunları içerir: konfigürasyon yükleme, bağımlılıkları kontrol etme, ağ bağlantısını başlatma, renderer'ı hazırlama, preview receiver'ı açma, frame'leri mobil cihaza gönderme, pose ve runtime durumunu güncelleme, ve son olarak sistemi temiz şekilde kapatma.

```
class Application {
public:
    Application();
    ~Application();

    int Run();

private:
    void StartHostSession();
    void StopHostSession();
    bool StartSteamVrRuntime();
    void StopSteamVrRuntime();

    renderer::RenderSystem render_system_;
    vr::VrRuntime vr_runtime_;
    tracking::PoseReceiver pose_receiver_;
    streaming::FrameStreamer frame_streamer_;
    std::unique_ptr<streaming::SharedFrameReceiver> preview_receiver_;
    network::UdpSocket telemetry_socket_;
    ui::DebugOverlay debug_overlay_;
    video::VideoPlayer video_player_;
    ViewMode view_mode_ = ViewMode::Video;
    video::VideoStereoLayout video_layout_ = video::VideoStereoLayout::SideBySide;
    video::VideoProjection video_projection_ = video::VideoProjection::Quad;
    bool video_flip_u_ = false;
    bool video_flip_v_ = false;
    bool video_load_requested_ = false;
    std::string video_load_path_;
    std::string steamvr_root_;
    std::string steamvr_driver_root_;
    std::string phone_stream_host_;
    std::string controller_status_ = "Idle";
    streaming::FrameStreamerConfig stream_config_;
    std::uint64_t last_streamed_preview_publish_count_ = 0;
    std::chrono::steady_clock::time_point last_streamed_preview_time_;
    std::chrono::duration<double> stream_frame_interval_(1.0 / 60.0);
    bool has_last_streamed_preview_time_ = false;
    bool host_session_running_ = false;
    bool start_session_requested_ = false;
    bool stop_session_requested_ = false;
};
```

2.4.2.1. Başlangıç Akışı

Application::Run() başladığında host sırasıyla şu kurulum adımlarını gerçekleştirir: WinSock başlatılır / bağımlılık bilgileri loglanır / theia_desktop.conf dosyasından konfigürasyon yüklenir / telefon stream host adresi çözülür / SteamVR root ve driver root dizinleri çözülür / stream konfigürasyonu hazırlanır (port, FPS, bitrate, payload boyutu) / telemetry socket açılır / VR runtime yapılandırılır / render sistemi başlatılır / debug overlay başlatılır

Bu sıra önemlidir çünkü host'un pasif bir renderer olmadığını gösterir. Oturum başlamadan önce tüm ortamı aktif şekilde hazırlar.

```
core::DependencyProbe::LogSummary(winsock.StartupResult());
const auto desktop_config = LoadDesktopConfig();
stream_config_.port = GetConfigInt(desktop_config, "stream_port", 9944);
stream_config_.target_fps = GetConfigInt(desktop_config, "stream_fps", 60);
stream_config_.bitrate_bps = GetConfigInt(desktop_config, "stream_bitrate_mbps", 20) * 1'000'000;
stream_config_.max_udp_payload_bytes = GetConfigInt(desktop_config, "stream_udp_payload_bytes", 1200);
phone_stream_host_ = ResolvePhoneStreamHost(desktop_config);
steamvr_root_ = ResolveSteamVrRoot(desktop_config);
steamvr_driver_root_ = ResolveSteamVrDriverRoot(desktop_config);
stream_frame_interval_ = std::chrono::duration<double>(1.0 / static_cast<double>(stream_config_.target_fps));

telemetry_socket_.Open(9750);
vr_runtime_.Configure({0.064F, 90.0F});
render_system_.Initialize("Theia Desktop Controller");
vr_runtime_.UpdateEyes(pose_receiver_.LatestPose(), render_system_.StereoAspectRatio());
controller_status_ = host_session_running_ ? "Streaming started" : "Ready";
debug_overlay_.Initialize(render_system_);
debug_overlay_.ShowStartupSummary(render_system_,
                                   vr_runtime_,
                                   frame_streamer_,
                                   pose_receiver_,
                                   telemetry_socket_,
                                   video_player_);

float previous_time = static_cast<float>(glfwGetTime());
```

2.4.2.2. Oturum Başlatma ve Durdurma

Host, oturum kontrolü için özel fonksiyonlara sahiptir: StartHostSession() / StopHostSession() / StartSteamVrRuntime() / StopSteamVrRuntime().

Oturum başladığında ise sırayla: SteamVR başlatılır veya hazırlanır, sürücü kaydedilir, pose receiver aktif hale getirilir, frame streamer yapılandırılır, shared preview receiver açılır, host running durumuna geçer.

Oturum durdurulduğunda ise: preview receiver kapatılır, frame streamer durdurulur, pose receiver durdurulur, SteamVR süreçleri sonlandırılır, driver kaydı kaldırılır. Bu temizlik önemlidir çünkü VR runtime'ları geride kalan driver durumlarına karşı hassastır. Host, sistemin her çalıştırmadan sonra temiz bir durumda kalmasını sağlar.

```
void Application::StopHostSession() {
    if (!host_session_running_) {
        return;
    }

    if (preview_receiver_) {
        preview_receiver_>Close();
        preview_receiver_.reset();
    }
    frame_streamer_.Shutdown();
    pose_receiver_.Stop();
    last_streamed_preview_publish_count_ = 0;
    has_last_streamed_preview_time_ = false;
    host_session_running_ = false;
    StopSteamVrRuntime();
    controller_status_ = "Streaming stopped";
    spdlog::info("Desktop host session stopped");
}
```

2.4.2.3. Ana Döngü Davranışı

Ana döngü içinde host her frame'de şu işlemleri tekrarlar:

- pencere event'lerini kontrol eder
- frame süresini hesaplar
- pose durumunu günceller
- preview receiver'ı günceller
- VR runtime'ı yeniler
- mevcut frame'i render eder
- debug overlay'i çizer
- start/stop isteklerini kontrol eder

```
debug_overlay_.BeginFrame();
debug_overlay_.Render(render_system_,
vr_runtime_,
frame_streamer_,
preview_receiver_.get(),
pose_receiver_,
telemetry_socket_,
video_player_,
view_mode_,
video_layout_,
video_projection_,
video_flip_u_,
video_flip_v_,
video_load_requested_,
video_load_path_,
host_session_running_,
controller_status_,
steamvr_root_,
steamvr_driver_root_,
start_session_requested_,
stop_session_requested_);
```

Bu döngü uygulamanın kalp atışıdır. Host'un SteamVR ve mobil istemci ile runtime durumunu sürekli senkronize ettiği yerdir.

2.4.2.4. Host'tan Mobile Frame Aktarımı

Host'un en önemli özelliklerinden biri, ham video verisini doğrudan göndermemesidir. Yeni bir preview frame oluşmasını bekler ve ayrıca yapılandırılmış frame aralığına (FPS) uyarak frame'i mobil cihaza iletir.

```
void Application::StartHostSession() {
    if (host_session_running_) {
        return;
    }

    if (!StartSteamVrRuntime()) {
        return;
    }

    pose_receiver_.Start();
    frame_streamer_.Configure(phone_stream_host_, stream_config_);
    preview_receiver_ = std::make_unique<streaming::SharedFrameReceiver>();
    preview_receiver_->Open();
    last_streamed_preview_publish_count_ = 0;
    has_last_streamed_preview_time_ = false;
    host_session_running_ = true;
    controller_status_ = "Streaming started";
    spdlog::info("Desktop host session started");
}
```

Pratikte bu şu şekilde gerçekleşir:

- display redirect driver preview frame üretir
- host bunları shared receiver üzerinden okur
- host FrameStreamer kullanarak mobil cihaza gönderir
- stream hızı yapılandırılmış FPS'e göre kontrol edilir

Dolayısıyla host, driver ile telefon arasında bir trafik yöneticisi gibi

davranır. Frame'lerin üretildiği yer değildir, ancak stream'in mobil cihaza devredildiği noktadır.

2.4.3. Renderer

Host'un görsel tarafı Desktop/src/Renderer/RenderSystem.h ve Desktop/src/Renderer/RenderSystem.cpp dosyaları tarafından yönetilir. Bu modül pencere oluşturma, OpenGL'i başlatma, shader pipeline'ı oluşturma, geometri oluşturma ve sahne ve video yüzeylerini render etme işlerinden sorumludur.

2.4.3.1. Bu Modül Ne Yapar?

Renderer, GLFW ve GLAD üzerine kuruludur. Masaüstü penceresini oluşturur, OpenGL context'i ayarlar, OpenGL fonksiyon pointer'larını yükler ve uygulama için gerekli GPU kaynaklarını hazırlar.

```
public:
    void Initialize(std::string_view window_title);
    void BeginFrame();
    void RenderScene(const vr::VrRuntime& vr_runtime,
                    const video::VideoPlayer& video_player,
                    video::VideoStereoLayout video_layout,
                    video::VideoProjection video_projection,
                    bool flip_u,
                    bool flip_v,
                    bool draw_scene,
                    bool draw_video);
    void RenderVideoFrame(const vr::VrRuntime& vr_runtime,
                        const video::VideoFrame& frame,
                        video::VideoStereoLayout video_layout,
                        video::VideoProjection video_projection,
                        bool flip_u,
                        bool flip_v);
    void EndFrame();
    void PollEvents();
    void RequestClose();
    void Shutdown();
```

2.4.3.2. How the Rendering Pipeline Works?

Kod iki shader pipeline tanımlar. Biri renkli sahne geometrisi için öbürü video yüzeyleri içindir.

Renderer ayrıca şu mesh olarak da sahne için küp, düz video için quad ve VR tarzı projeksiyon için küre ve yarım küre hazırlar.

Bu sayede host hem genel bir 3D ortamı hem de gelen video stream'ini uygun projeksiyon moduna göre render edebilir.

Ancak genel olarak SteamVR zaten kendi geometrisini oluşturduğu için bu yapı sistemi gereksiz yere karmaşık hale getirebilir. Bu yüzden pratikte SteamVR'ın render'ı kullanılır ve ham görüntü doğrudan gösterilir.

```
void RenderSystem::InitializeWindow(const std::string_view window_title) {
    if (!glfwInit()) {
        throw std::runtime_error("glfwInit failed");
    }

    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_DOUBLEBUFFER, GLFW_TRUE);

    int window_width = 1280;
    int window_height = 820;
    int window_x = GLFW_DONT_CARE;
    int window_y = GLFW_DONT_CARE;

    if (GLFWMonitor* monitor = glfwGetPrimaryMonitor(); monitor != nullptr) {
        int monitor_x = 0;
        int monitor_y = 0;
        int monitor_width = 0;
        int monitor_height = 0;
        glfwGetMonitorWorkarea(monitor, &monitor_x, &monitor_y, &monitor_width, &monitor_height);
        if (monitor_width > 0 && monitor_height > 0) {
            window_width = std::clamp(static_cast<int>(monitor_width * 0.34F), 1180, 1680);
            window_height = std::clamp(static_cast<int>(monitor_height * 0.46F), 760, 1140);
            window_x = monitor_x + (monitor_width - window_width) / 2;
            window_y = monitor_y + (monitor_height - window_height) / 2;
        }
    }
}
```

2.4.3.3. Video Yerleşim Yönetimi

Renderer şu video yerleşimlerini destekler: yan yana (side-by-side) stereo / alt-üst (over-under) stereo / düz veya küresel projeksiyon / UV çevirme. Bu yapı başlangıçta SteamVR'ın zaten raw render verdiği bilinmediği için geliştirilmiştir. Ancak yine de sistemin yapısını anlamamı sağlamış ve projeyi SteamVR dışına genişletme fikrini uyandırmıştır.

```
if (video_layout == theia::video::VideoStereoLayout::SideBySide) {
    transform.scale_x = 0.5F;
    transform.offset_x = (viewport_x == 0) ? 0.0F : 0.5F;
} else if (video_layout == theia::video::VideoStereoLayout::OverUnder) {
    transform.scale_y = 0.5F;
    transform.offset_y = (viewport_x == 0) ? 0.0F : 0.5F;
}

if (flip_u) {
    transform.offset_x += transform.scale_x;
    transform.scale_x = -transform.scale_x;
}
if (flip_v) {
    transform.offset_y += transform.scale_y;
    transform.scale_y = -transform.scale_y;
}
```

2.4.3.4. Bu Dosya Neden Önemli?

Bu modül host uygulamasının görsel temelidir. Olmadan pencere olmaz, render context olmaz, stream gösterimi olmaz. Ayrıca OpenGL başlatma ve shader kurulumunu içerdiği için ekran görüntüsü alınabilecek en uygun yerlerden biridir.

2.4.4. Debug ve Kontrol Arayüzü

Kullanıcıya görünen kontrol paneli Desktop/src/UI/DebugOverlay.h ve Desktop/src/UI/DebugOverlay.cpp dosyalarında uygulanmıştır. Bu katman host'u etkileşimli hale getirir. ImGui kullanılarak oluşturulmuştur ve masaüstü uygulamasının ana kontrol paneli görevini görür.

2.4.4.1. Ne Gösteriyor?

Overlay üç sekmeden oluşur: General, Streaming ve Tracking. Her sekme farklı bir runtime alanına odaklanır. General, oturum durumu ve yolları halleder. Streaming, frame ve stream istatistiklerini halleder. Tracking ise pose ve tracking durumunu halleder.

Bu kısım rapor için önemlidir çünkü host'un sadece arka planda çalışan bir servis olmadığını, canlı teşhis bilgileri sunan bir kontrol aracı olduğunu gösterir.


```

if (ImGui::BeginTabBar("controller-tabs", ImGuiTabBarFlags_NoCloseWithMiddleMouseButton)) {
    if (ImGui::BeginTabItem("General")) {
        active_tab = 0;
        ImGui::EndTabItem();
    }
    if (ImGui::BeginTabItem("Streaming")) {
        active_tab = 1;
        ImGui::EndTabItem();
    }
    if (ImGui::BeginTabItem("Tracking")) {
        active_tab = 2;
        ImGui::EndTabItem();
    }
    ImGui::EndTabBar();
}

```

2.4.4.2. Neden Önemli?

Overlay, sistemin “nasıl düşündüğünü” görmeyi sağlar. Stream durumu, renderer, pose verisi ve SteamVR ayarlarını tek bir yerde gösterir.

2.4.5. Streaming Aktarımı

Host’tan mobile veri gönderimi Desktop/src/Streaming/FrameStreamer.cpp dosyasında yapılır. Arayüzü Desktop/src/Streaming/FrameStreamer.h içindedir. Bu sınıf, hazırlanmış video frame’lerini UDP üzerinden Android cihaza göndermekten sorumludur.

2.4.5.1. Peki Ne Yapar?

Streamer: hedef host ve port bilgisini tutar, UDP socket oluşturur, encoder’ın hazır olmasını sağlar, gelen frame’i uygun formata dönüştürür ve encode edilmiş veriyi mobil cihaza gönderir.

Burda önemli nokta Host’un ham texture göndermemesi ve önce encode edilip, sonra paketlenme yapılmasıdır.

```

void FrameStreamer::SendFrame(const video::VideoFrame& frame) {
    if (!IsConfigured() || frame.width <= 0 || frame.height <= 0 || frame.rgb_pixels.empty()) {
        return;
    }
    if (!EnsureSocket() || !EnsureEncoder(frame.width, frame.height)) {
        return;
    }

    const std::uint8_t* source_data[4] = {frame.rgb_pixels.data(), nullptr, nullptr, nullptr};
    const int source_linesize[4] = {frame.width * 4, 0, 0, 0};
    sws_scale(sws_context_,
              source_data,
              source_linesize,
              0,
              frame.height,
              frame->data,
              frame->linesize);
}

```

2.4.5.2. Neden Önemli?

Bu, sistemi cihazlar arası yapan katmandır. Masaüstü gönderici, telefon alıcıdır. Bu ilişki burada gerçekleşir.

2.4.6. Pose ve Tracking Durumu

```
#include "Tracking/HeadPose.h"

namespace theia::tracking {

class PoseReceiver {
public:
    void Start();
    void Stop();
    void Update(bool move_forward,
                bool move_backward,
                bool move_left,
                bool move_right,
                bool turn_left,
                bool turn_right,
                bool look_up,
                bool look_down,
                float delta_seconds);

    [[nodiscard]] bool IsRunning() const noexcept;
    [[nodiscard]] HeadPose LatestPose() const noexcept;

private:
    bool is_running_ = false;
    HeadPose latest_pose_{{0.0F, 1.6F, 0.0F}, {0.0F, 0.0F, 0.0F}};
};
```

Host tarafındaki pose mantığı Desktop/src/Tracking/PoseReceiver.h ve PoseReceiver.cpp dosyalarında bulunur.

Bu sınıf:

- mevcut kafa pozisyonunu saklar
- klavye ile hareket edildiğinde günceller
- pitch değerini sınırlar (mantıksız dönmeyi engeller)

2.4.6.1. Ne Yapar?

Receiver; pozisyon, yaw, pitch ve roll değerlerini tutar. Bunları kullanarak Update fonksiyonunu çağırır. Bu şekilde Update fonksiyonu ileri ve geri hareketi, sola ve sağa hareketi, sola ve sağa dönmeyi ve yukarı ve aşağı bakmayı desteklemiş oluyor.

Host, bu pose durumunu VR runtime güncellemesinin bir parçası olarak kullanır. Mevcut mimaride pose receiver, hareket ve yönelim durumunun host tarafındaki temsilidir.

```
void PoseReceiver::Update(const bool move_forward,
                           const bool move_backward,
                           const bool move_left,
                           const bool move_right,
                           const bool turn_left,
                           const bool turn_right,
                           const bool look_up,
                           const bool look_down,
                           const float delta_seconds) {

    if (!is_running_) {
        return;
    }
}
```

2.4.6.2. Neden Önemli?

Bu kısım, host'un sadece frame göndermediğini gösterir. Aynı zamanda VR runtime'a ve debug arayüzüne beslenen canlı bir tracking durumu da tutar.

3. Mobil

3.1. Project Teknolojileri

3.1.1. Modül(ler)

- **Android (Android API 24+):**
Projenin Android tarafı Android 13 ve üzeri sürümlerde çalışacak şekilde tasarlanmıştır. Android 7 ve üzeri cihazlarda çalışmaması için bir neden yoktur, ancak test edilecek bir cihaz bulunmadığı için bu durum belirsiz kabul edilmektedir.

3.1.2. Programlama Dili

- **Kotlin – Android Studio:**
Proje Kotlin ile yazılmıştır.

3.1.3. Kütüphaneler ve Harici Bileşenler

- **androidx.core:core-ktx**
Kotlin dostu Android çekirdek uzantıları ve yardımcı fonksiyonlar sağlar.
- **androidx.lifecycle:lifecycle-runtime-ktx**
Activity ve composable akışı için lifecycle farkındalıklı runtime desteği sağlar.
- **androidx.activity:activity-compose**
Compose UI'nin ComponentActivity içinde barındırılmasını sağlar.
- **androidx.compose.ui.graphics**
Compose tabanlı renk ve çizim desteği sağlar.
- **androidx.compose.ui.tooling.preview**
Compose önizleme ve geliştirme desteği sağlar.
- **androidx.compose.material3**
Material 3 tasarım sistemi ve metin stilini sağlar.
- **android.media.MediaCodec**
Cihaz üzerinde donanımsal hızlandırılmış H.264 çözme (decode) için kullanılır.
- **android.media.MediaFormat**
Decoder formatını ve düşük gecikmeli oynatma ayarlarını yapılandırmak için kullanılır.
- **android.view.Surface / SurfaceView**
Decode edilen video stream'ini ekranda göstermek için kullanılır.
- **android.hardware.SensorManager**
Rotasyon sensörlerine ve cihaz hareket verilerine erişmek için kullanılır.
- **Jetpack Compose BOM**
Compose UI bağımlılıklarını senkron tutmak için kullanılır.

3.2. Uygulama Kabuk Yapısı

Mobil uygulamanın giriş noktası Mobile/app/src/main/java/com/example/theia/MainActivity.kt dosyasıdır. Bu dosya arayüzü bilinçli olarak minimal şekilde tasarlanmıştır. Sonuçta normalde kullanılmayacak. Activity: edge-to-edge modu aktif eder, ekranın açık kalmasını sağlar, sistem çubuklarını gizler, video render için bir SurfaceView açar, surface yaşam döngüsüne bağlı olarak streaming oturumunu başlatır veya durdurur. Uygulama ayrıca çift dokunma ile açılıp kapatılabilen küçük bir debug overlay içerir. Bu overlay şu bilgileri gösterir: receiver durumu, decoder durumu, pose durumu, yerel IP adresi, stream ve pose portları. Ekranı çift tıklama ile açılır.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        keepScreenOn()
        enterImmersiveMode()

        setContent {
            TheiaTheme {
                Surface(modifier = Modifier.fillMaxSize(), color = Color.Black) {
                    TheiaStreamScreen()
                }
            }
        }
    }
}
```

Bu yapı, ayrı bir ayarlar ekranına ihtiyaç duymadan Android uygulamasının kolayca test edilmesini sağlar.

3.3. Oturum Koordinasyonu

Oturum yöneticisi app/src/main/java/com/example/theia/streaming/TheiaStreamSession.kt'dir.

Bu sınıf üç streaming bileşeni arasında bağlayıcı görevi görür: video receiver / H.264 decoder / pose sender. Surface hazır olduğunda start(...) fonksiyonu önce decoder'ı oluşturur, sonra UDP receiver'ı oluşturur, peşine pose sender'ı başlatır ve son olarak tüm worker thread'leri başlatır.

Surface yok edildiğinde stop() fonksiyonu tüm sistemi temiz şekilde kapatır.

Bu tasarım Android tarafını basitleştirir çünkü Activity her bileşeni ayrı ayrı yönetmek zorunda kalmaz. Oturum nesnesi yaşam döngüsünü yönetir ve ağ/çözme mantığını tek yerde toplar.

```
fun start(surface: Surface) {
    stop()

    val h264Decoder = H264SurfaceDecoder(
        width = videoWidth,
        height = videoHeight,
        surface = surface,
        onStatus = onDecoderStatus,
    )

    val udpReceiver = UdpH264Receiver(
        port = port,
        onSample = h264Decoder::queueSample,
        onStatus = { status ->
            poseSender.updateTargetHost(status.streamHost)
            onReceiverStatus(status)
        },
    )

    decoder = h264Decoder
    receiver = udpReceiver
    h264Decoder.start()
    poseSender.start()
    udpReceiver.start()
}
```

3.4. Video Alma

Gelen video stream'i Mobile/app/src/main/java/com/example/theia/streaming/UdpH264Receiver.kt dosyasında işlenir. Bu sınıf UDP port 9944 üzerinde dinleme yapar ve paket parçalarından encode edilmiş video örneklerini yeniden oluşturur.

Her UDP paketinin tam bir frame içerdiğini varsaymaz. Bunun yerine önce, özel protokol header'ını parse eder, sonra paket geçerliliğini kontrol eder, peşine paketleri sample ID'ye göre gruplar ve son olarak sample tamamlanana kadar parçaları birleştirir.

```
private fun receiveLoop() {
    try {
        val datagramSocket = DatagramSocket(port).apply {
            receiveBufferSize = 4 * 1024 * 1024
        }
        socket = datagramSocket
        publishStatus( state = "Listening on UDP $port")

        val buffer = ByteArray( size = 1600)
        while (isRunning.get()) {
            val packet = DatagramPacket(buffer, length = buffer.size)
            datagramSocket.receive( p = packet)
            streamHost = packet.address?.hostAddress
            handlePacket(packet.data, packet.length)
        }
    } catch (_: SocketException) {
        if (isRunning.get()) {
            publishStatus( state = "Socket closed unexpectedly")
        }
    } catch (exception: Exception) {
        publishStatus( state = "Receiver error: ${exception.message ?: exception::class.java.simpleName}")
    } finally {
        socket?.close()
        socket = null
    }
}
```

Yani receiver gerçek anlamda yeniden yapılandırma işlemi yapar:

- header doğrulaması
- geçersiz paketleri yok sayma
- duplicate ve eski paketleri takip etme
- parçaları birleştirerek tam sample oluşturma
- uzun süren eksik sample'ları temizleme

Bu önemli bir kısımdır çünkü UDP güvenilir değildir, bu yüzden istemci stream'i dikkatli şekilde yeniden oluşturmak zorundadır.

3.5. Video Çözme

Decode işlemi Mobile/app/src/main/java/com/example/theia/streaming/H264SurfaceDecoder.kt dosyasında gerçekleştirilir. Bu sınıf Android MediaCodec kullanarak gelen H.264 stream'ini doğrudan bir Surface üzerine çözer. Bu yaklaşım düşük gecikmeli bir görüntüleyici için uygundur çünkü ekstra render katmanlarını ortadan kaldırır ve mobil donanım üzerinde daha verimli çalışır.

Decoder ise sırayla H.264 codec oluşturur, düşük gecikme ayarları ile yapılandırır, encode edilmiş veriyi input buffer'lara besler, decode edilen frame'leri surface'e aktarır, küçük bir queue tutarak akıcı bir akış sağlar. Queue dolarsa eski sample'lar düşürülebilir. Bu, uygulamanın gecikme yerine güncelliği koruması için yapılan bilinçli bir tercihtir.

```
private fun decodeLoop() {
    try {
        val mediaCodec = MediaCodec.createDecoderByType(MediaFormat.MIMETYPE_VIDEO_AVC)
        val format = MediaFormat.createVideoFormat( mime = MediaFormat.MIMETYPE_VIDEO_AVC, width, height).apply {
            setInteger( name = MediaFormat.KEY_LOW_LATENCY, value = 1)
        }
        mediaCodec.configure(format, surface, crypto = null, flags = 0)
        mediaCodec.start()
        codec = mediaCodec
        publishStatus( state = "Decoder ready")

        val bufferInfo = MediaCodec.BufferInfo()
        while (isRunning.get()) {
            feedInput(mediaCodec)
            drainOutput(mediaCodec, bufferInfo)
        }
    } catch (exception: Exception) {
        publishStatus( state = "Decoder error: ${exception.message ?: exception::class.java.simpleName}")
    } finally {
        codec?.runCatching {
            stop()
            release()
        }
        codec = null
    }
}
```

3.6. Pose Gönderimi

Pose tarafı Mobile/app/src/main/java/com/example/theia/streaming/PhonePoseSender.kt dosyasında uygulanmıştır. Bu sınıf telefon sensörlerinden hareket verisini okur ve yönelim paketlerini UDP üzerinden masaüstü host'a gönderir. Cihaza bağlı olarak game rotation vector veya rotation vector sensörünü kullanır.

```
private fun sendLoop() {
    while (isRunning.get()) {
        val packetBytes = latestPacketBytes
        val target = targetAddress
        val currentSocket = socket
        if (packetBytes != null && target != null && currentSocket != null) {
            runCatching {
                val packet = DatagramPacket( buf = packetBytes, length = packetBytes.size, address = target, port)
                currentSocket.send( p = packet)
            }.onFailure { exception ->
                onStatus("Pose send error: ${exception.message ?: exception::class.java.simpleName}")
            }
        }
        Thread.sleep( millis = 8L)
    }
}
```

Bu kısım telefon ile masaüstü arasındaki döngüyü tamamlar. Mobil uygulama yalnızca görüntü hedefi değildir; aynı zamanda sisteme yönelim verisi sağlayan bir bileşendir.

3.7. Protokol Katmanı

Mobil taraf ayrıca özel protokol tanımlarını içerir:

- Mobile/app/src/main/java/com/example/theia/streaming/VideoStreamProtocol.kt
- Mobile/app/src/main/java/com/example/theia/streaming/PoseStreamProtocol.kt

Bu dosyalar:

- magic değerleri
- versiyon numaralarını
- header boyutlarını
- frame / sequence ID'leri
- payload boyutlarını

tanımlıyor.

Bu küçük ama önemli bir katmandır çünkü host ve mobil tarafın güvenilir şekilde iletişim kurabilmesi için aynı paket yapısını kullanmaları gerekir.

```
val frameId = buffer.long
val sampleId = buffer.long
val packetIndex = buffer.int
val packetCount = buffer.int
val payloadSize = buffer.int
val flags = buffer.int
if (packetCount <= 0 || packetIndex < 0 || packetIndex >= packetCount || payloadSize < 0) {
    return null
}
if (HeaderSize + payloadSize > length) {
    return null
}

return Header(
    frameId = frameId,
    sampleId = sampleId,
    packetIndex = packetIndex,
    packetCount = packetCount,
    payloadSize = payloadSize,
    flags = flags,
)
```

4. Proje Analizi

4.1. Test

Testler kişisel bilgisayarımda ve telefonumda yapıldı. Sistem 30 dakikalık zaman aralıklarında test edildi. Gecikme ve paket kaybı düşüktü. Yani tüm testlerin başarılı olduğunu söyleyebiliriz.

4.2. Geliştirilebilir Şeyler

Şu an, Theia yalnızca SteamVR'ye bağlı olarak çalışabiliyor ve tek başına bir anlam sunamıyor. İlerleyen zamanlarda bu projeyi herhangi bir platformdan bağımsız olarak geliştirip kullanıcılara sağlam çalışmalar sunabiliriz.

5. Proje Yorumları

5.1. Kullanırken Yapılacaklar

Masaüstü uygulamasını telefonun IP'sine göre yapılandırmayı unutmayın. Mobil uygulamada hata ayıklama kullanıcı arayüzünü çift tıklayıp açarak bulabilirsiniz.

5.2. Yapımcı Yorumları

Bu benim ilk Turuncu proje fikrimdi. Janus ben VR isteyip de param olmadığı için ortaya çıkabildi. Bu fikir aklıma hiç gelmeseydi böyle şeylerle hiç uğraşmış ve hiç öğrenmemiş olurum.O yüzden bu projeyi seviyorum.

Hooah

-Famura

6. Katkıda Bulunanlar

- Sadece Famura projesi

7. Referanslar

- SteamVR
Valve Corporation. (n.d.). *SteamVR*. <https://store.steampowered.com/steamvr>
- OpenVR
Valve Corporation. (n.d.). *OpenVR SDK*. <https://github.com/ValveSoftware/openvr>
- FFmpeg
FFmpeg Developers. (n.d.). *FFmpeg*. <https://ffmpeg.org/>
- x264
VideoLAN. (n.d.). *x264 H.264 encoder*. <https://www.videolan.org/developers/x264.html>
- Media Foundation
Microsoft. (n.d.). *Microsoft Media Foundation*. <https://learn.microsoft.com/en-us/windows/win32/medfound/microsoft-media-foundation>
- OpenGL
Khronos Group. (n.d.). *OpenGL*. <https://www.opengl.org/>
- GLFW
GLFW. (n.d.). *GLFW*. <https://www.glfw.org/>
- GLAD
GLAD. (n.d.). *GLAD OpenGL loader*. <https://glad.dav1d.de/>
- GLM
G-Truc Creation. (n.d.). *GLM (OpenGL Mathematics)*. <https://github.com/g-truc/glm>

- Direct3D 11
Microsoft. (n.d.). *Direct3D 11*. <https://learn.microsoft.com/en-us/windows/win32/direct3d11/direct3d-11-graphics>
- Android Studio
Google. (n.d.). *Android Studio*. <https://developer.android.com/studio>
- Android MediaCodec
Google. (n.d.). *MediaCodec API*. <https://developer.android.com/reference/android/media/MediaCodec>
- Android SensorManager
Google. (n.d.). *SensorManager*. <https://developer.android.com/reference/android/hardware/SensorManager>
- Jetpack Compose
Google. (n.d.). *Jetpack Compose*. <https://developer.android.com/jetpack/compose>
- CMake
Kitware. (n.d.). *CMake*. <https://cmake.org/>
- Dear ImGui
Ocornut. (n.d.). *Dear ImGui*. <https://github.com/ocornut/imgui>
- spdlog
Gabi Melman. (n.d.). *spdlog*. <https://github.com/gabime/spdlog>
- stb
Barrett, S. (n.d.). *stb libraries*. <https://github.com/nothings/stb>
- Windows API
Microsoft. (n.d.). *Windows API*. <https://learn.microsoft.com/en-us/windows/win32/api>
- Win32 API
Microsoft. (n.d.). *Win32 API*. <https://learn.microsoft.com/en-us/windows/win32/>
- H.264
ITU-T. (2019). *Advanced video coding for generic audiovisual services (H.264)*.
- UDP
Postel, J. (1980). *User Datagram Protocol*. RFC 768. <https://doi.org/10.17487/RFC0768>
- Winsock
Microsoft. (n.d.). *Windows Sockets 2 (Winsock)*. <https://learn.microsoft.com/en-us/windows/win32/winsock/windows-sockets-start-page>

8. Disk İçeriği

- Bu raporun .docx formatındaki hali
- Bu raporun .pdf formatındaki hali
- Projenin kaynak kodları
- Projenin Android release'i
- Projenin Windows release'i
- Elektrisini Orgazam'dan Igra Rock'en'Roll Cela Jugoslavia